

PDO Troubleshooting, Security Essentials, and MySQLi (Optional)

Learning Goals

- Diagnose and resolve common PDO connection failures (missing driver, stopped services, bad credentials)
- Explain the purpose of prepared statements in separating SQL structure from user data (preventing SQL injection)
- Distinguish database security (prepared statements) from browser output safety (`htmlspecialchars()`)
- Compare emulated prepares (`ATTR_EMULATE_PREPARES => true`) with native prepares (`false`)
- Recognize MySQLi syntax and contrast its workflow (`bind_param`) with PDO (optional reference)

Start Here If Something Fails

Troubleshooting is the first part of this document.

Check these in order:

1. PDO MySQL driver
2. PHP and MySQL server status
3. Connection information: database, username, and password

Check the PDO MySQL Driver

If `new PDO("mysql:...")` reports **could not find driver**:

```
php -m | grep -i pdo  
php -i | grep "PDO drivers"
```

Ubuntu / WSL2:

```
sudo apt install php-mysql
```

macOS (Homebrew PHP):

PDO MySQL is typically built-in. If not recognized, verify your active PHP binary and loaded modules:

```
which php  
php --ini
```

Restart the PHP server after installing or updating modules.

Check the Two Servers

If PHP cannot connect to MySQL, check each server separately.

1. Is the PHP development server running?
2. Is the MySQL service running?
3. Does the database name match `studentdb`?
4. Do the username and password match your MySQL account?

WSL2/Ubuntu:

```
sudo service mysql status
```

macOS:

```
brew services list
```

IMPORTANT

Security Essentials and Reference

With troubleshooting covered, this document addresses two critical areas:

1. Security Essentials (Core):

- Why prepared statements prevent SQL injection
- Why `htmlspecialchars()` protects HTML output from XSS

2. Optional Deep Dive & Reference:

- Emulated vs native prepares
(`ATTR_EMULATE_PREPARES`)

Why Use Prepared Statements?

A query has two different parts: the SQL structure and the changing value.

- The value may come from a user, such as a student ID entered in a form.

```
SQL structure: SELECT * FROM students WHERE id = :id  
Value:         :id receives 7 (for example)
```

`:id` is a named empty place in the SQL structure.
PDO supplies the value only when `execute()` runs.

Prepared statements keep these parts separate.

- This prevents user input from altering the SQL command structure, neutralizing **SQL injection attacks**.
- It also lets PDO safely handle values containing quotes (such as `0' Connor`).

For example, if an input provides the ID `7`:

```
$stmt = $pdo->prepare(  
    "SELECT * FROM students WHERE id = :id"  
);  
$stmt->execute([':id' => 7]);
```

`prepare()` creates a prepared statement object.

- With native prepares, PDO sends the SQL structure to MySQL for preparation; with emulated prepares, PDO manages the preparation behavior itself.
- `execute()` supplies `7` as data for that placeholder.

Safe HTML Output

Database safety and browser output safety are different concerns.

- Prepared statements protect the SQL query when PHP saves or reads a value.
- They do not control how the browser displays that value.

For example, a student name stored in MySQL might contain `<b>Alice</b>`; without escaping, the browser treats it as HTML markup instead of displaying literal text.

```
echo htmlspecialchars($student['name'], ENT_QUOTES, 'UTF-8');
```

- `htmlspecialchars()` converts HTML-special characters to safe entities before output, preventing Cross-Site Scripting (XSS).
- Both prepared statements and output escaping are essential layers in secure web applications.

Deep Dive:

ATTR_EMULATE_PREPARES (Optional)

This is one optional setting added when PHP creates the `$pdo` connection:

```
$pdo = new PDO($dsn, $username, $password, [
    PDO::ATTR_EMULATE_PREPARES => false,
]);
```

Both choices (true/false) run the same PHP code, but they send the query to MySQL differently:

Setting	What happens	Advantage	Limitation
true	PDO quotes and binds <code>:id</code> locally as data, then sends the SQL.	Default; simple for standard CRUD examples.	MySQL does not prepare the SQL structure separately.
false	PDO sends SQL structure and <code>:id</code> separately.	MySQL parses and prepares the SQL on the server.	Can behave differently for some advanced queries.

| When **false** Is Useful: Bulk Operations

For example, an application imports 10,000 students using the same **INSERT** command:

```
$stmt = $pdo->prepare(  
    "INSERT INTO students (name) VALUES (:name)"  
);  
$stmt->execute([':name' => 'Alice']);  
$stmt->execute([':name' => 'Bob']);  
// Repeat for remaining students
```

- With `false`, MySQL prepares the `INSERT` structure **once** on the server and executes it repeatedly with new values, avoiding repeated query parsing.
- For standard web requests, performance differences are minor and depend on query patterns and network round-trips.

| Why Core Examples Use `true` (Default)

The core code does not set this option, so `pdo_mysql` uses its default, `true`.

- With `true`, the PDO driver safely quotes and binds parameter values into the SQL query locally before sending the complete statement to MySQL.
- Because values are properly quoted as data, they cannot alter the SQL structure.
- We use this default to focus first on the `prepare()` and `execute()` pattern without requiring extra connection settings.

MySQLi Reference (Optional)

MySQLi is another PHP interface for MySQL.

- It only supports MySQL/MariaDB.
- It uses `?` placeholders and a separate `bind_param()` step.
- You may encounter it in existing PHP code.
- This course teaches and uses PDO instead.

See `insert_mysqli_optional.php` (first INSERT)
and
`crud_mysqli_optional.php` (full CRUD) in
`module1/code/2_Connect_PHP_with_MySQL/` for
working examples.

Required: PDO
Reference only: MySQLi

PDO and MySQLi Syntax

PDO:

```
$stmt = $pdo->prepare(  
    "SELECT * FROM students WHERE id = :id"  
);  
$stmt->execute([':id' => $aliceId]);  
$student = $stmt->fetch();
```

MySQLi reference:

```
$stmt = $conn->prepare(  
    "SELECT * FROM students WHERE id = ?"  
);  
$stmt->bind_param("i", $aliceId);  
$stmt->execute();
```